CIS220 Database Concepts
Individual Assignment
The Relational Database Model

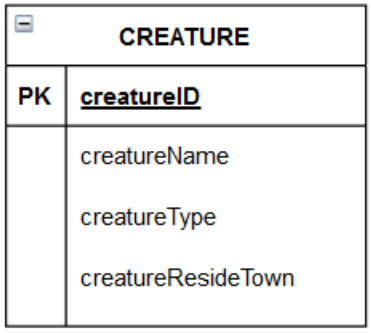
Objectives

- Demonstrate how relationships function within the Relational Model
- Define tables with foreign key constraints
- Create explicit relationships between tables
- Write SQL queries that join multiple tables

Assignment: As you work through this assignment you will be asked to create screenshots. Create a document and add each of the screen shots in the order they are requested.

When you have finished, submit the document as a PDF (please follow this instruction)

Let's continue the Creature case study and examine how relationships shown on a conceptual model (schema) can actually be implemented using relational database tables. So far we have worked with a database containing a single table.



Suppose there was more data that we wanted to remember about each town besides its name. For example, the town of Bemidji, MN, USA has a nickname and motto, as seen on its wikipedia page. We have two approaches to consider.

1. Add the extra details (attributes) to the Creature table

   OR

2. Create a table to store all of the specific data about towns

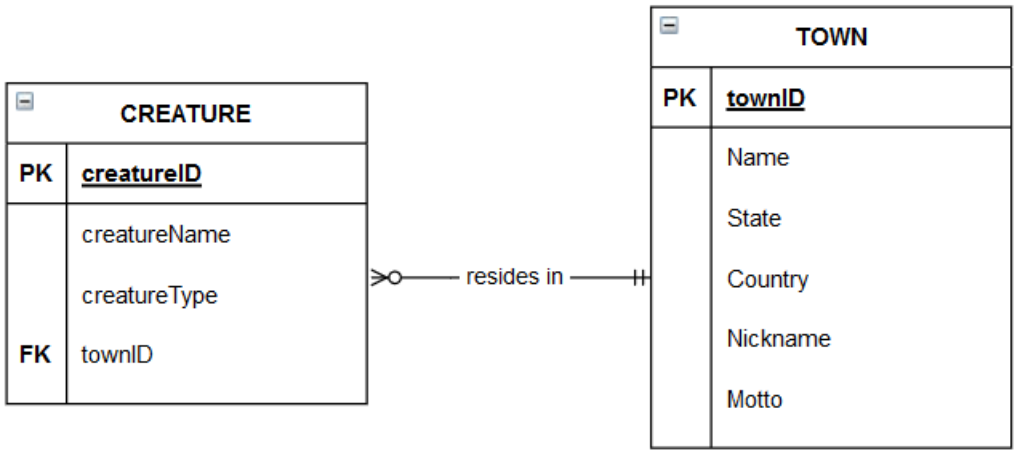
Question: Which is the correct design?

Answer: Number 2. Adding the Town data into the Creature table would be poor design because it would cause ***redundancy***. You would need to duplicate the town data **for every creature** that lives in that town.

Example: If you wanted to store the nickname for Bemidji, MN ("The First City on the Mississippi") in the Creature table you would have to enter it for every creature that resides in this town. This redundancy has the potential to cause inaccuracies if the nickname ever changes or if someone just randomly decides to enter a different nickname or misspell.

Best Practice: Only store non-primary key attribute values in the database **ONCE** and then create relationships whenever you need to link those values to other contexts.

This would cause us to make this evolution of our conceptual model:



Changes:

1. Added the TOWN table
2. Changed the CREATURE attribute ***creatureResideTown VARCHAR*** to ***townID INT***
3. Made CREATURE.townID a ***FOREIGN KEY that REFERENCES TOWN.townID***
4. It's important to understand that the data type must also change to match the data type of the attribute it references . . . which is TOWN.townID

We now have a 1:M relationship between the tables CREATURE and TOWN. This relationship can be expressed in the following way

- A creature resides in **one** town and only one town. If the creature relocates we would update the townID to reflect the change. If the creature relocated to a new town that does not exist in our database, then we would have to ***add the town first*** so we would have a primary key to reference with our foreign key.
- A town can have **many** creatures residing there. Multiple creatures could reside in 'Gotham City'. Gotham City's townID could appear **many** times in the Creature table, once for each creature that resides there.
- The technique for implementing a 1:M relationship is always the same. Take the primary key from the "one side" (Town) of the relationship and place it as a foreign key in the "many side" (Creature)

TASK ONE:

- You have been provided with the following SQL file: **creature.town.sql**
- This file contains the table definition and insert statements for the Town table described above.
- Study the contents of this file to ensure you understand what is happening. Notice the alternate notation for primary key.
- **Using the MySQL Monitor terminal interface:** Source this script file into the **creature_demo** database you created for last week's assignment. The database will now have 2 ***unrelated*** tables
- **SUBMISSION: Using the MySQL Monitor terminal interface:** Take a screen shot of the table description and show the full contents of the table and add it to the submission document

TASK TWO:

- You now need to modify the structure of the CREATURE table to reflect our changes in design. Do the following:
- Copy the original CREATE TABLE statement for the Creature table into your current ***creature.town.sql*** script. It needs to be placed **after** the creation and data insertion of the town table because it will reference the Town table in its CREATE TABLE statement, therefore that table must exist and be populated already or you will get errors.
- Modify the CREATE TABLE statement for the CREATURE table to reflect the changes described above. Include a foreign key constraint.
- Here is an example of a ***foreign key constraint*** from the **Classic Models** database that creates a relationship between the Customer and Employee tables

    - CONSTRAINT customers_fk_1
      FOREIGN KEY (salesRepEmployeeNumber)
      REFERENCES employees (employeeNumber)

- **NOTES:**
    - The constraint must be given an identifier so the database engine can make a note of it
    - The ***foreign key*** field is named ***salesRepEmployeeNumber***
    - The ***foreign key*** field gets its value from the referenced field ***employeeNumber*** in the table ***employees***

- Copy over the CREATURE INSERT INTO statements from last week's script and modify the insertions so that the creature instances link to the proper town. The insert statements now need to contain appropriate ***townID*** values.
- Now your script file should have the following structure
    1. Conditionally create the database
    2. Creation of the Town table
    3. Insertion of Town data
    4. Creation of Creature table with foreign key constraint
    5. Insertion of Creature data
- Can you explain why the Town table ***must*** be created and populated first? This may be a test question.

- Once this is done, **Using the MySQL Monitor terminal interface:** take a screen shot showing
    - the creature database tables
    - the structure of the Creature table
    - All of the Creature data
    - the structure of the Town table
    - All of the Town data

TASK THREE: Write SQL INNER JOIN statements to answer the following questions. You are required to use the search criteria that I specify in the question. ***No shortcuts by looking it up yourself in the database.*** Pretend there are millions of records in these tables and it would be impossible to look it up by scrolling through the data.

The queries need to use the search criteria that are included in each problem . . . nothing else.

SUBMISSION: Screen shot the query results and *be sure to include the SQL code in the screen shot*.

1. Show all Creature names that live in 'United States'. Use United States as the search criteria.
2. Show all creature names that do not live in the 'United States'
3. Show the motto for the town that The Tick resides in. Use The Tick as the search criteria
4. Show all the "person" creatures that live in Minnesota (MN). Use 'person' and 'MN' as the search criteria.
5. How many creatures live in 'The City'? Use the COUNT function and 'The City' as the search criteria
6. Show all the creature types that live in 'The City'. Use 'The City' as the search criteria

TASK FOUR: SQL Insert statements and relationships

1. Try to insert a new creature into the creature table. Choose a town that does not exist. What happens when this query runs? Do you know why? Include your answer to this question in the submission document.
2. Insert a new town of your choosing. Include data for all fields.
3. Insert a new creature that resides in this town
4. Show the creature name, creature type, town name and town motto for the new creature

SUBMISSION: Screen shot the SQL results results and *be sure to include the SQL code in the screen shot*. Export your solution document to a PDF file and submit the PDF. Review the instructions carefully to ensure you have completed everything correctly.